Keystone Connect Production Technical Blueprint - Executive Summary

My Goal

I am building Keystone Connect as a reliable contractor discovery platform that helps users search by location and construction category, review results quickly, save preferred companies, remove irrelevant results, and export usable project-ready data.

My goal is to keep the user experience simple while making the system underneath stable, secure, and scalable.

What Keystone Connect Is

Keystone Connect is a search-driven contractor discovery system built around:

- location-based searching,
- CSI-style construction categories,
- multiple search modes for different levels of strictness,
- persistent preferred and irrelevant result handling,
- and downloadable project-friendly exports.

It is not just a directory. I am treating it as a controlled contractor intelligence tool.

How It Works Today

Today, Keystone Connect runs as a Node.js application that serves both the UI and backend.

Its core pieces are:

- a web server,
- a search engine,
- a structured category taxonomy,
- a desktop/mobile UI,
- and lightweight JSON persistence for preferred and suppressed results.

Searches are powered by Google Places and processed through ranking, filtering, deduplication, and radius enforcement before results are returned to the user.

What Makes the Product Useful

The product is useful because it gives users a practical workflow:

1. Enter a location
2. Choose a trade or category
3. Select a radius
4. Search
5. Review and manage results
6. Export for follow-up work

The system also supports:

- tighter search mode,
- broader search mode,
- prior-result hiding during radius expansion,
- preferred result saving,
- irrelevant result removal,
- and `.xlsx` export generation.

What I Need for Production

To make Keystone Connect production-grade, I need to move from prototype-grade persistence and workflows into a more controlled architecture.

That means:

- replacing runtime JSON persistence with PostgreSQL,
- adding automated testing,
- adding structured logging and monitoring,
- validating API payloads,
- protecting search endpoints with rate limits,
- improving caching,
- and formalizing deployment and rollback.

My Production Architecture Direction

My production target is intentionally simple:

1. A Node web/API service
2. PostgreSQL for persistent shared data
3. Optional async workers for slower enrichment tasks
4. Basic observability for logs, metrics, and errors

This keeps the system manageable while removing the biggest operational risks.

My Data Direction

I want Keystone Connect to become the system of record for:

- search runs,
- preferred companies,
- suppressed companies,
- project-specific memory,
- and future team workflow status.

I do not want shared spreadsheets to become the live database. I want spreadsheets to remain exports, while the app owns the truth.

My Quality Standard

I want the app to be dependable in real use.

That means:

- stable UI behavior across desktop and mobile,
- predictable search behavior,
- no silent failures,
- partial-result handling when upstream providers fail,
- and regression testing against known cities and business types.

My Release Standard

I want releases to be disciplined and repeatable.

A production-ready release should include:

- automated checks,
- package validation,

- health checks,
- clean desktop build outputs,
- and a clear rollback path.

My Long-Term Direction

Long term, I want Keystone Connect to support:

- project-based contractor lists,
- shared internal workflows,
- cleaner no-duplicate expansion searching,
- and generated master workbooks by division without spreadsheet conflicts.

Bottom Line

I am building Keystone Connect as a practical internal production tool: easy to use on the front end, but engineered underneath so it can support repeatable searching, better contractor review workflows, stronger data quality, and future team-scale operations.